VerifyDoc Readme

Field	Value
Version	1.0
Status	Draft
Last Updated	14/10/2026
Owner	VerifyDoc

Navigation

- VerifyDoc Readme
  - Navigation
  - Overview
  - Project Status
  - Features
  - Not Included in the MVP
  - Tech Stack
  - Architecture
  - Getting Started
    - Prerequisites
    - Environment Variables
    - Scripts
    - Testing
  - API Overview
  - Roles and Statuses
  - Core Rules
  - Security
  - Known Limitations
  - Documentation
  - Team

Overview

VerifyDoc is a private, role-based digital credential verification platform for universities, polytechnics, bootcamps, employers, and citizens. This repository contains the backend REST API that powers the VerifyDoc MVP.

In VerifyDoc, approved institutions issue credentials, citizens store them in a private wallet and choose who can see them, and approved employers verify the credentials shared with them.

Project Status

The backend MVP is implemented and documented. Web and mobile integration, real-user acceptance testing, and production deployment checks are still to come, so this README does not describe a customer-facing launch.

Features

- Registration and login for citizens, employer organizations, and institutions, with JWT authentication
- Email verification and a password reset flow that uses one-time codes
- Admin approval of employer organizations and institutions before they can use protected features
- Credential issuance by approved institutions
- Citizen credential upload, reviewed by the issuing institution (verify or reject)
- Credential revocation by the issuing institution
- A private citizen wallet with filtering and pagination
- QR code generation and public credential verification through a signed token
- Credential sharing: citizens share one credential at a time with an approved employer, set a purpose and expiry (up to 365 days), and can revoke a share
- Employer access to shared credentials, with private document links left out
- Fresh confirmation: employers ask the issuing institution to reconfirm a credential, and the institution responds Confirmed or Disputed

Not Included in the MVP

- Search for citizens by name, email, or phone number
- Audit logs and notifications
- Advanced fraud detection and AI or OCR document tamper detection
- Live integrations with external bodies (for example NIMC, WAEC, or CAC)
- Payments or subscriptions
- Account deletion

Tech Stack

- Node.js and Express
- MongoDB (MongoDB Atlas) with Mongoose
- JSON Web Tokens (JWT) and bcrypt
- An email service for password-reset and verification codes
- A QR code library
- Cloudinary for document storage (integration not finalized)

Architecture

A request passes through these layers in order:

Routes → Middleware → Controllers → Services → Models → MongoDB

Layer	Responsibility
Routes	Map endpoints to middleware and controllers
Middleware	Handle JWT authentication, roles, approval status, ownership, request validation, uploads, and errors
Controllers	Read request input, call services, and send responses
Services	Hold business logic and database workflows
Models	Define MongoDB schemas, validation, and relationships
Utils	Provide helpers such as credential ID generation, QR token generation, and OTP hashing

Getting Started

Follow these steps to run the API locally.

Prerequisites

You need Node.js, npm, and access to a MongoDB database.

Installation

1. Clone the repository and move into it:

```
git clone <repository-url>
 cd VerifyDoc-backend-system
```

2. Install dependencies:

```
npm install
```

3. Create a `.env` file in the project root and add the variables listed below.
4. Start the development server:

```
npm run dev
```

5. Send `GET /` to `http://localhost:5000` (or your configured port). The server replies with the plain text `Server is running`.

Environment Variables

Set these in your `.env` file. Never commit real values.

Variable	Description
`PORT`	The port the server listens on (for example `5000`)
`MONGODB_URI`	Your MongoDB connection string
`JWT_SECRET`	A long, random secret used to sign access tokens
`JWT_EXPIRES_IN`	Access token lifetime (for example `1d`)

Variable	Description
`FRONTEND_URL`	The frontend origin used to build QR verification links
`QR_VERIFICATION_SECRET`	The secret used to sign QR verification tokens
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Document storage credentials (integration not finalized)
`RESEND_API_KEY`, `EMAIL_FROM`	Email delivery for one-time codes

Scripts

Command	Purpose
`npm run dev`	Start the server in development mode
`npm start`	Start the server in production mode
`npm test`	Run the automated tests

Testing

The backend includes automated tests, including database-backed integration tests.

API Overview

The API has 41 endpoints. Paths are relative to your deployed API origin, for example `http://localhost:5000/auth/login` locally. Protected endpoints require the header `Authorization: Bearer <access_token>`. Tokens last 1 day.

Resource	Example paths	Endpoints
Authentication and users	`/auth/register`, `/auth/login`, `/auth/me`, `/users/me`	10
Onboarding	`/auth/register-organization`, `/auth/register-institution`	2

Admin	`/admin/approval-queue`, `/admin/organizations/:id/approve`	9
Credentials and wallet	`/credentials`, `/credentials/wallet`	7
QR verification	`/credentials/:id/qr`, `/credentials/verify/:id`	2
Credential sharing	`/credential-shares`, `/credential-shares/mine`	6
Fresh confirmation	`/credential-shares/organization/:id/confirmations`	4
Health check	`/`	1

For full request and response details, see the VerifyDoc API Documentation.

Roles and Statuses

VerifyDoc has four roles:

Role	Main responsibility
Citizen	Views their own credentials, uploads credentials for review, and shares them with approved employers
Employer organization	Views credentials shared with it and requests fresh confirmation
Institution	Issues credentials, reviews citizen uploads, revokes credentials, and responds to confirmation requests
Platform Admin	Approves or rejects employer organizations and institutions

Credential and sharing records use these status values:

Record	Field	Values
Credential	`verificationStatus`	`Pending`, `Verified`, `Rejected`
Credential	`status`	`Active`, `Revoked`

Share	`status`	`Active`, `Expired`, `Revoked`, `Unavailable`
Confirmation request	`status`	`Pending`, `Confirmed`, `Disputed`

Core Rules

- Only approved institutions can issue, verify, reject, or revoke credentials.
- Only approved employer organizations can receive shared credentials or request fresh confirmation.
- Citizens see only their own credentials, and employers can't search for citizens.
- Employers see a credential only while an active, unexpired share exists, and their view leaves out the private document link.
- A citizen-uploaded document stays `Pending` until the issuing institution verifies it.
- Admin accounts can't be created through public registration.
- Credentials are revoked, not deleted, and are corrected by revoking and reissuing.

Security

- Passwords are hashed with bcrypt, and password-reset codes are hashed and expire.
- Access uses JWTs, with role-based authorization, approval status checks, ownership checks, and request validation on protected actions.
- Secrets are read from environment variables, never from source code.
- Production hardening is still in progress, including a refresh-token policy, network-level protections, and backup and recovery.

For details, see the Security Architecture Overview.

Known Limitations

- Institution registration currently fails with `400 Invalid role`, and a related verification-email bug follows once that is fixed.
- Replacing a profile picture can leave the old image uncleaned.
- There is no refresh-token mechanism, so users log in again after 1 day.
- Only the public verification endpoint has rate limiting.
- No endpoint exists to delete an account.
- A production base URL, Cloudinary integration, and production CORS rules are not finalized.

For the full list, see Section 7 of the API Documentation.

Documentation

- VerifyDoc API Documentation
- Security Architecture Overview
- Data Privacy Policy
- Citizen User Guide
- Employer Verification Guide
- Institutions Onboarding Guide
- FAQ

Team

DEVS:

- Koto Temiloluwa John
- Ogunyemi Michael

WRITERS:

- Chibuikem Iwuh
- Dada Marcus